In overtested projects, even the simplest change takes too long to complete. Either seemingly unrelated tests start failing, or the test suite takes too long to run for developers to be productive.

Not all tests are worth the effort it takes to write and maintain them. In fact, some tests have *negative value*. Kent Beck's Stack Overflow answer on this topic is instructive here:

> I get paid for code that works, not for tests, so my philosophy is to test as little as possible to reach a given level of confidence....[3]

We recommend periodically taking stock of long-running projects. Are basic logic bugs making it into production? Consider shoring up those areas of your test suite. Are bogus failures getting in the way of code changes? Try modifying your specs to be less brittle, or even deleting some of them.

As your architecture changes, tests can become superfluous. Deleting a test doesn't mean any of the effort writing it was wasted or misspent. It simply means it's overstayed its welcome. A test that can't pay for itself shouldn't be in your suite any more.

Deciding What *Not* to Test

Every behavior you specify in a test is another point of coupling between your tests and your project code. That means you'll have one more thing you'll have to fix if you ever need to change your implementation's behavior.

Sometimes, it's better to decide intentionally *not* to test certain things. For instance, user interfaces can change rapidly. If you couple your automated tests tightly to incidental details of your UI, you increase the cost of change. You may get more mileage out of manual exploratory testing—see Elisabeth Hendrickson's *Explore It! [Hen13]* for more information on this topic.

If you do need to drive a UI from automated tests, try to test in terms of your problem domain ("log in as an administrator") rather than implementation details ("type `admin@example.com` into the third text field").

Another key place to show restraint is the level of detail in your test assertions. Rather than asserting that an error message exactly matches a particular string ("Could not find user with ID 123"), consider using substrings to match just the key parts ("Could not find user"). Likewise, don't specify the exact order of a collection unless the order is important.

3. https://stackoverflow.com/questions/153234/how-deep-are-your-unit-tests/153565#153565